

Question 1 (EASY)

Approach -

1. Input Handling

- Read integer N
- Read N space-separated integers

2. Merge sort logic

- use divide and conquer.
- Recursively split the array into halves.
- sort each half
- Merge the two sorted halves.

3. Merge Function

- compare elements from left and right subarrays.
- place the smaller element into the main array.
- continue until elements are merged.

4. output

print the sorted array in ascending order

```
#include <iostream>
```

```
#include <vector>
```

```
using namespace std;
```

```
void merge(vector<int> &arr, int left, int mid, int right) {
```

```
    int n1 = mid - left + 1;
```

```
    int n2 = right - mid;
```

```
    vector<int> L(n1), R(n2);
```

```
    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
```

```
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];
```

```
    int i = 0, j = 0, k = left;
```

```
    while (i < n1 & j < n2) {
```

```
        if (L[i] <= R[j]) {
```

```
            arr[k++] = L[i++];
```

```
        } else {
```

```

else {
    arr[k++] = R[j++];
}
while (L[i] < n1) arr[k++] = L[i++];
while (L[j] < n2) arr[k++] = R[j++];
while (L[i] < n1) arr[k++] = L[i++];
while (L[j] < n2) arr[k++] = R[j++];
}
void mergesort (vector<int> &arr, int left, int right) {
    if (left < right) {
        int mid = (left + (right - left) / 2);
        mergesort (arr, left, mid);
        mergesort (arr, mid + 1, right);
        merge (arr, left, mid, right);
    }
}
int main () {
    int N;
    cout << "Enter Number of Students: ";
    cin >> N;
    vector<int> package (N);
    cout << "Enter package value (multiplied by 100)";
    for (int i = 0; i < N; i++) {
        cin >> package[i];
    }
    mergesort (package, 0, N - 1);
    for (int i = 0; i < N; i++) {
        cout << package[i] << " ";
    }
    cout << endl;
    return 0;
}

```

```

PS D:\STUDY\DSA_c++\3_week_module> cd "d:\STUDY\DSA_c++\3_week_module\" ; if ($?) { g++ Question1_Topic1.cpp -o Question1_Topic1 } ; if ($?) { .\Question1_Topic1 }
Enter Number of Students: 6
Enter Package values (multiplied by 100): 450 1200 800 650 300 1500
300 450 650 800 1200 1500
PS D:\STUDY\DSA_c++\3_week_module>

```

Question 2 (Medium):

Approach -

1. Input: Read N
2. Sorting: use Merge sort to sort the times
3. Median:
 - If N is odd $\rightarrow$ middle element
 - If N is even $\rightarrow$ average of two middle elements
4. Count above median: Traverse sorted array and count values greater than median.
5. Difference - subtract minimum (first element) from maximum (last element)

```
#include <iostream>
```

```
#include <vector>
```

```
using namespace std;
```

```
void merge(vector<int> &arr, int left, int mid, int right) {
```

```
    // same
```

```
void mergeSort(vector<int> &arr, int left, int right) {
```

```
    // same
```

```
int main() {
```

```
    int N;
```

```
    cout << "Enter Number of order: ";
```

```
    cin >> N;
```

```
    vector<int> times(N);
```

```
    cout << "Enter processing times: ";
```

```
    for (int i = 0; i < N; i++) {
```

```
        cin >> times[i];
```

```
    }  
    mergeSort(times, 0, N-1);
```

```

for(int i=0; i<N; i++){
    cout << times[i] << " ";
}

```

```

2
cout << endl;

```

```

double median;

```

```

if (N%2 == 2){

```

```

    median = times[N/2];

```

```

} else{

```

```

    median = (times[N/2 - 1] + times[N/2]) / 2;

```

```

    cout << "median: " << median << endl;

```

```

    int countAbove = 0;

```

```

    for(int i=0; i<N; i++){

```

```

        if (times[i] > median) countAbove++;
    }

```

```

    cout << "orders Above Median: " << countAbove;

```

```

    int difference = times[N-1] - times[0];

```

```

    cout << "Difference: " << difference << endl;

```

```

    return 0;
}

```

```

PS D:\STUDY\DSA_c++\3_week_module> cd "d:\STUDY\DSA_c++\3_week_module\" ; if ($?) { g++ Question2_Topic1.cpp -o Question2_Topic1 } ; if ($?) { .\Question2_Topic1 }
Enter Number of Order: 7
Enter Processing times: 12 25 18 30 15 22 40
12 15 18 22 25 30 40
Median: 22
Orders Above Median: 3
Difference: 28
PS D:\STUDY\DSA_c++\3_week_module>

```

Topic 2: Quick sort

Question 1 (EASY)

Approach

1. Input : Read N

2. Quick sort:

- choose a pivot (here, last element)
- partition the array so that smaller elements go left, larger go right
- Recursively sort left and right partitions.

3. output : Print sorted response times in ascending order

```
#include <iostream>
```

```
#include <vector>
```

```
int partition (vector<int> &arr, int low, int high) {
```

```
    int pivot = arr[high];
```

```
    int i = low - 1;
```

```
    for (int j = low; j < high; j++) {
```

```
        if (arr[j] <= pivot) {
```

```
            i++;
```

```
            swap(arr[i], arr[j]);
```

```
        }
```

```
        swap(arr[i+1], arr[high]);
```

```
        return i+1;
```

```
    }
```

```
void quicksort (vector<int> &arr, int low, int high) {
```

```
    if (low < high) {
```

```
        int pi = partition(arr, low, high);
```

```
        quicksort(arr, low, pi-1);
```

```
        quicksort(arr, pi+1, high);
```

```
    }
```

```

int main()
{
    int N;
    cout << "Enter Number of response Time:";
    cin >> N;
    vector<int> responseTimes(N);
    cout << "Enter response Time:";
    for (int i = 0; i < N; i++)
    {
        cin >> responseTimes[i];
    }
    quickSort(responseTimes, 0, N-1);
    for (int i = 0; i < N; i++)
    {
        cout << responseTimes[i] << " ";
    }
    cout << endl;
    return 0;
}

```

```

PS D:\STUDY\DSA_c++\3_week_module> cd "d:\STUDY\DSA_c++\3_week_module\" ; if ($?) { g++ Question1_Topic2.cpp -o Question1_Topic2 } ; if ($?) { .\Question1_Topic2 }
Enter Number of response Time: 6
Enter responseTime: 120 45 78 200 60 95
45 60 78 95 120 200
PS D:\STUDY\DSA_c++\3_week_module>

```

Question 2 (MEDIUM):

Approach

1. Input Handling

- Read integer N
- Read N space-separated trade values into a vector.

2. Quick sort implementation

- use a recursive Quick sort algorithm.
- Modify the partition function so that values greater than pivot are placed on left
- Recursively sort the left and right partitions

3. Display sorted value

- After sorting, print the entire array in descending order

4. Top 5 values.

- since the array is sorted descending, the first 5 elements are the top 5 trade values.
- print them directly

5. Average Top 5

6. overall Average

7. count values greater than overall Average

#include <iostream>

#include <vector>

#include <iomanip>

int partition (vector<long long>& arr, int low, int high) {

long long pivot = arr[high]

int i = low - 1;

for (int j = low; j < high; j++) {

if (arr[j] >= pivot) {

i++;

Topic 3: Heap sort

Question 1 (Easy):

1. Input Handling

- Read integer N
- Read N space-separated scores into a vector

2. Heap sort implementation

- Build a max-heap from the scores
- Repeatedly extract the minimum element and place it at the end of the array
- This produces a sorted array in ascending order after the full process

3. Output

- Print the sorted in ascending order

```
#include <iostream>
```

```
#include <vector>
```

```
using namespace std;
```

```
void heapify (vector<int>& arr, int n, int i) {
```

```
    int largest = i;
```

```
    int left = 2*i + 1;
```

```
    int right = 2*i + 2;
```

```
    if (left < n & arr[left] > arr[largest])
```

```
        largest = left;
```

```
    if (right < n & arr[right] > arr[largest])
```

```
        largest = right;
```

```
    if (largest != i) {
```

```
        swap (arr[i], arr[largest]);
```

```
        heapify (arr, n, largest);
```

```
    }
```

Healthy (arr 1, 0)

Return 0)

PS D:\STUDY\DSA c++\3 week module>

Question 2 (Medium)

Approach -

1. Input Handling

- Read integer N
- Read N space-separated response times into a vector.

2. Heap sort Implementation.

- Build a max-heap.
- Extract elements one by one to produce ascending order.

3. Statistics calculation

- Sorted response time $\rightarrow$ print the array
- Fastest response time $\rightarrow$ first element of sorted array
- slowest response time $\rightarrow$ last element of sorted array
- Average response time $\rightarrow$ sum of all times $\div N$
- Cases faster than average $\rightarrow$ count element strictly less than average
- per centage faster than average $\rightarrow$ $(\text{count} / N) * 100$

#include <iostream>

#include <vector>

#include <algorithm>

using namespace std;

void heapify (vector<int> &arr, int n, int i)

{
 void heapsort (vector<int> &arr, int n) {

int main() {

```

int N;
cin >> N;
vector<int> times(N);
for(int i=0; i<N; i++){
    cin >> times[i];
}
heap sort (times, N);

for(int i=0; i<N; i++){
    cout << times[i] << " ";
}
cout << endl;

int fastest = times[0];
int slowest = times[N-1];

long long sum=0;
for(int i: times) sum += i;
double avg = (double)sum/N;
int countFaster = 0;
for(int i: times){
    if(t < avg) countFaster++;
}
double percentage = (double)countFaster * 100.0 / N;

cout << "Fastest:" << fastest << endl;
cout << "Slowest:" << slowest << endl;
cout << "Average:" << fixed << setprecision(2) << avg << endl;
cout << "Cases Faster Than Average:" << countFaster << endl;
cout << "percentage" << fixed << setprecision(2) << percentage << "%." << endl;

return 0;
}

```

```

PS D:\STUDY\DSA_c++\3_week_module> cd "d:\STUDY\DSA_c++\3_week_module\" ; if ($?) { g++ Question2_Topic3.cpp -o Question2_Topic3 } ; if ($?) { .\Question2_Topic3 }
8
12 7 15 9 20 5 10 8
5 7 8 9 10 12 15 20
Fastest: 5
Slowest: 20
Average: 10.75
Cases Faster Than Average: 5
Percentage: 62.50%
PS D:\STUDY\DSA_c++\3_week_module>

```